

PassPort 백엔드

Postman 사용 매뉴얼

현재 단계(1단계: 엔티티 · 임시 CRUD API) 기준

작성일 2026-07-02 · 작성자 원석(BE)

이 문서는 코딩을 몰라도 따라 할 수 있도록 만들었습니다. Postman은 "서버에 요청을 보내고 응답을 눈으로 확인하는 도구"이고, 여기서는 우리 프로젝트가 만든 8개의 임시 API를 실제로 눌러보면서 확인하는 방법을 다룹니다.

0. 먼저 알아야 할 것

이 API들은 아직 로그인 기능이 없는 임시 버전입니다. 누구나 인증 없이 호출할 수 있고, 2단계(인증) 작업이 끝나면 요청마다 로그인 토큰을 함께 보내야 하는 방식으로 바뀝니다. 지금은 "백엔드가 실제로 동작하는지", "화면에서 어떤 데이터를 주고받게 될지"를 미리 확인하는 용도로 사용해주세요.

서버를 켜면 테스트용 회원(User) 2명이 자동으로 미리 등록되어 있습니다 (id=1: demo1@passport.ac.kr, id=2: demo2@passport.ac.kr). 회원가입 API가 아직 없기 때문에, 이 두 계정의 번호(userId)를 이용해서 프로필을 만드는 것부터 테스트를 시작합니다.

1. 준비물

1-1. Postman 설치

데스크톱 앱을 아직 설치하지 않았다면 postman.com 공식 사이트에서 무료로 내려받아 설치합니다 (회원가입은 구글 계정으로 간단히 가능).

1-2. 서버 실행 (백엔드 담당자 또는 IntelliJ에서)

아래 둘 중 하나의 방법으로 서버를 켭니다. 서버가 켜지지 않으면 Postman에서 아무 요청도 성공하지 않습니다.

방법	설명
IntelliJ에서 실행	PassportApplication.java 파일을 열고 좌측 초록색 ▶ 버튼(main 메서드 옆) 클릭
터미널에서 실행	프로젝트 폴더에서 gradlew bootRun 명령 실행 (Windows는 gradlew.bat bootRun)

콘솔(실행 로그)에 Started PassportApplication 이라는 줄이 뜨면 준비가 끝난 것입니다. 서버 주소는 기본적으로 http://localhost:8080 입니다.

2. Postman 컬렉션 불러오기

매번 주소와 요청을 손으로 입력할 필요 없이, 이미 만들어진 파일 하나를 불러오면 8개의 요청이 폴더별로 정리되어 나타납니다.

단계	화면에서 할 일
①	Postman 왼쪽 위 Import 버튼 클릭
②	프로젝트 폴더의 postman/PassPort.postman_collection.json 파일을 선택 (드래그 앤 드롭도 가능)
③	왼쪽 사이드바(Collections)에 PassPort API (임시 - 인증 불기 전) 항목이 생기면 성공

2-1. 변수(Variables) 확인

컬렉션에는 매번 바뀌는 값을 미리 이름 붙여둔 "변수"가 들어있습니다. URL에 {{profileId}}처럼 중괄호로 표시된 부분이 변수입니다. 컬렉션 이름 위에 마우스를 올려 점 3개(⋮) → Edit → Variables 탭에서 값을 바꿀 수 있습니다.

변수명	기본값	의미
baseUrl	http://localhost:8080/api/v1	서버 기본 주소. 포트를 바꿨다면 여기만 수정하면 전체 요청에 적용됨
userId	1	프로필을 만들 때 연결할 회원 번호 (시드된 1 또는 2 사용)
profileId	1	만들어진 프로필의 번호. 처음엔 임의값이지만 프로필 생성 후 실제 id로 바꿔줘야 함
courseId	1	만들어진 수강 이력의 번호. 수강 이력 등록 후 실제 id로 바꿔줘야 함

3. 실습 순서 (이 순서대로 눌러보세요)

아래 순서대로 진행하면 회원 → 프로필 → 수강 이력 → 졸업 인증까지 전체 흐름을 한 번에 체험할 수 있습니다.

순서	화면	요청	확인 포인트
1	Profile 폴더 → 프로필 생성	POST /profiles	userId=1로 프로필 생성. 응답의 data.id 값을 확인 (보통 1)
2	컬렉션 변수 profileId를 방금 확인한 값으로 수정	-	이후 모든 요청이 이 프로필을 기준으로 동작함
3	Profile 폴더 → 프로필 조회	GET /profiles/{id}	방금 만든 프로필 정보가 그대로 나오는지 확인
4	Profile 폴더 → 프로필 수정	PATCH /profiles/{id}	이름 등을 바꿔서 보내고, 응답에 반영됐는지 확인
5	Course 폴더 → 수강 이력 등록	POST /profiles/{id}/courses	과목 하나를 등록. 응답의 data.id를 courseId 변수에 반영
6	Course 폴더 → 수강 이력 조회/수정/삭제	GET · PUT · DELETE	등록한 과목이 목록에 보이는지, 수정·삭제가 반영되는지 확인
7	Certification 폴더 → 졸업 인증 조회	GET /profiles/{id}/certifications	처음에는 빈 배열 []이 정상
8	Certification 폴더 → 졸업 인증 갱신	PUT /profiles/{id}/certifications	어학·봉사·논문 상태를 보내고, 다시 조회해서 반영 확인

각 요청 화면 오른쪽 위의 Send 버튼을 누르면 아래쪽에 서버 응답(JSON)이 나타납니다. 상태 코드가 200이나 201이면 성공입니다.

4. 엔드포인트별 상세 (요청 · 응답 예시)

4-1. Profile

● 프로필 생성

POST /profiles

회원(userId)에 학과·학번 정보를 연결해서 프로필을 만듭니다. 회원 1명당 프로필은 1개만 만들 수 있습니다.

요청 예시 (HTTP 201)

```
{
  "userId": 1,
  "deptCode": "BIGDATA_AI",
  "studentId": "2021123456",
  "admissionYear": 2021,
  "name": "홍길동"
}
```

응답 예시 (HTTP 201)

```
{
  "success": true,
  "data": {
    "id": 1,
    "userId": 1,
    "deptCode": "BIGDATA_AI",
    "studentId": "2021123456",
    "admissionYear": 2021,
    "name": "홍길동"
  },
  "message": null
}
```

● 프로필 조회

GET /profiles/{id}

id로 프로필 정보를 조회합니다.

응답 예시 (HTTP 200)

```
{
  "success": true,
  "data": { "id": 1, "userId": 1, "deptCode": "BIGDATA_AI",
    "studentId": "2021123456", "admissionYear": 2021, "name": "홍길동" },
  "message": null
}
```

● 프로필 수정

PATCH /profiles/{id}

학과·학번·이름 등을 수정합니다.

요청 바디

```
{
  "deptCode": "BIGDATA_AI",
  "studentId": "2021123456",
  "admissionYear": 2021,
  "name": "김철수"
}
```

응답 예시 (HTTP 200)

```
{
  "success": true,
  "data": { "id": 1, "userId": 1, "deptCode": "BIGDATA_AI",
            "studentId": "2021123456", "admissionYear": 2021, "name": "김철수" },
  "message": null
}
```

4-2. Course (수강 이력)

● 수강 이력 등록

POST /profiles/{id}/courses

과목명·학점·이수구분·성적·연도·학기를 등록합니다. category는 MAJOR_REQUIRED/MAJOR_ELECTIVE/GE_REQUIRED/GE_ELECTIVE/GENERAL_ELECTIVE 중 하나, grade는 "A+"처럼 문자열로 보냅니다.

요청 바디

```
{
  "name": "자료구조",
  "credit": 3,
  "category": "MAJOR_REQUIRED",
  "grade": "A+",
  "year": 2023,
  "semester": 1
}
```

응답 예시 (HTTP 201)

```
{
  "success": true,
  "data": { "id": 1, "name": "자료구조", "credit": 3,
            "category": "MAJOR_REQUIRED", "grade": "A+", "year": 2023, "semester": 1 },
  "message": null
}
```

● 수강 이력 조회

GET /profiles/{id}/courses

해당 프로필에 등록된 모든 수강 이력을 배열로 반환합니다.

요청 예시 (HTTP 200)

```
{
  "success": true,
  "data": [
    { "id": 1, "name": "자료구조", "credit": 3, "category": "MAJOR_REQUIRED",
      "grade": "A+", "year": 2023, "semester": 1 }
  ],
  "message": null
}
```

● 수강 이력 수정

PUT /profiles/{id}/courses/{cid}

특정 과목 하나를 통째로 새 값으로 덮어씁니다.

요청 예시

```
{
  "name": "자료구조",
  "credit": 3,
  "category": "MAJOR_REQUIRED",
  "grade": "A",
  "year": 2023,
  "semester": 1
}
```

응답 예시 (HTTP 200)

```
{
  "success": true,
  "data": { "id": 1, "name": "자료구조", "credit": 3,
    "category": "MAJOR_REQUIRED", "grade": "A", "year": 2023, "semester": 1 },
  "message": null
}
```

● 수강 이력 삭제

DELETE /profiles/{id}/courses/{cid}

특정 과목을 삭제합니다. 성공하면 data는 없습니다(null).

요청 예시 (HTTP 200)

```
{
  "success": true,
  "data": null,
  "message": null
}
```

4-3. Certification (졸업 인증)

● 졸업 인증 조회

GET /profiles/{id}/certifications

어학·봉사·논문 인증 상태를 조회합니다. 아직 한 번도 갱신하지 않았다면 빈 배열이 정상입니다.

요청 예시 (HTTP 200)

```
{
  "success": true,
  "data": [],
  "message": null
}
```

● 졸업 인증 갱신

PUT /profiles/{id}/certifications

type(LANGUAGE/VOLUNTEER/THESIS)별 status(PASS/FAIL/NOT_SUBMITTED)를 한 번에 여러 개 보낼 수 있습니다. 이미 있는 type이면 상태만 바뀌고, 없던 type이면 새로 생깁니다.

요청 예시

```
{
  "items": [
    { "type": "LANGUAGE", "status": "PASS" },
    { "type": "VOLUNTEER", "status": "NOT_SUBMITTED" },
    { "type": "THESIS", "status": "FAIL" }
  ]
}
```

요청 예시 (HTTP 200)

```
{
  "success": true,
  "data": [
    { "type": "LANGUAGE", "status": "PASS" },
    { "type": "VOLUNTEER", "status": "NOT_SUBMITTED" },
    { "type": "THESIS", "status": "FAIL" }
  ],
  "message": null
}
```


5. 자주 만나는 에러와 대처법

증상	원인	해결
Could not send request / Connection refused	서버가 꺼져 있음	IntelliJ 또는 터미널에서 서버를 다시 실행
404, code: PROFILE-001	존재하지 않는 profileId를 사용함	먼저 "프로필 생성"을 실행해서 실제 id를 확인하고 변수에 반영
409, code: PROFILE-002	이미 그 회원(userId)의 프로필이 있음	userId를 2로 바꿔서 새로 만들거나, 이미 만든 프로필의 id를 그대로 사용
404, code: AUTH-004	존재하지 않는 userId(1, 2 외의 번호)를 사용함	userId를 1 또는 2로 되돌리기
500, 서버 내부 오류	요청 바디 JSON 형식이 잘못됨	Body 탭이 raw + JSON으로 되어 있는지, 따옴표·심표가 맞는지 확인

6. 보너스 — DB에 실제로 저장됐는지 눈으로 확인하기

브라우저에서 `http://localhost:8080/h2-console` 접속 → JDBC URL에 `jdbc:h2:mem:passportdb` 입력(사용자명 `sa`, 비밀번호는 비워둠) → Connect. 왼쪽에서 `USERS`, `PROFILES`, `COURSES`, `CERTIFICATIONS` 테이블을 눌러보면 Postman으로 만든 데이터가 실제 표(테이블) 형태로 보입니다.

이 DB는 메모리에만 저장되는 임시 DB라서, 서버를 껐다 켜면 데이터가 전부 초기화됩니다 (테스트용 회원 2명만 다시 자동 생성됩니다). 실제 서비스에서 쓸 DB는 아직 팀에서 확정되지 않았습니다.

여기까지 따라 하셨다면 백엔드 API 8개가 전부 정상 동작하는 것을 직접 확인하신 겁니다.